

DEPARTAMENTO DE INFORMÁTICA

INF2102 — PROJETO FINAL DE PROGRAMAÇÃO

Verificação Simbólica e Auditável de Princípios Éticos em Agentes Baseados em Foundation Models

ai-ethical-guardrail

Gabriel Mariquito
gsousa@inf.puc-rio.br

Professora:
Greis Silva-Calpa

Rio de Janeiro
Agosto de 2026

Conteúdo

1	Breve Descrição	3
1.1	A principal função do programa	3
1.2	Funções específicas que o programa oferece	3
1.3	Usuários primordialmente contemplados	4
1.4	A natureza do programa	4
2	Visão de Projeto	4
2.1	Cenário Positivo 1 (i.e. cenário que dá certo)	4
2.2	Cenário Positivo 2	4
2.3	Cenário Negativo 1 (i.e. cenário que expõe uma limitação conhecida e esperada do programa)	4
2.4	Cenário Negativo 2	5
3	Documentação Técnica do Projeto	5
3.1	Requisitos funcionais	5
3.2	Requisitos não funcionais	6
3.3	Arquitetura e organização do software	6
3.4	Sobre o código	10
4	Considerações finais	14
5	Manual de Utilização para Usuários Contemplados	15
5.1	Tarefa A — Instalação	15
5.2	Tarefa B — Abrir a interface (com o programa já instalado)	16
5.3	Tarefa C — Checar um conteúdo e ler a sua explicação	16
5.4	Tarefa D — Conversar com um modelo protegido pelo guardrail	17
5.5	Tarefa E — Rodar a demonstração	18
5.6	Tarefa F — Executar a avaliação	18
5.7	Tarefa G — Ler a trilha de auditoria	19
5.8	Tarefa H — Proteger um modelo em código próprio	20
6	Referências	21
7	Apêndice — como reconferir os números deste documento	21

1 Breve Descrição

1.1 A principal função do programa

O `ai-ethical-guardrail` é um guardrail simbólico para Foundation Models, bilíngue, escrito em Python. Cada entrada é avaliada antes de chegar ao modelo, e cada resposta antes de ser entregue. O processo de verificação é composto de duas camadas, uma que opera sobre o texto e outra que opera sobre conceitos. As duas camadas produzem vereditos independentes, e o mais restritivo prevalece.

O filtro permite quatro respostas, dentre elas:

- i) **Allow**, a resposta passa sem alteração para o agente/usuário;
- ii) **Flag**, a resposta passa, mas fica anotada para revisão posterior;
- iii) **Rewrite**, parte do conteúdo é transformada ou redigida; e
- iv) **Deny**, a resposta é bloqueada por ferir alguma regra/norma.

1.2 Funções específicas que o programa oferece

Funções específicas do programa:

- Suporte a auditoria. Uma interface para auditores apresenta, para cada decisão, a norma que decidiu com seu princípio e justificativa, o trecho de texto que a ativou, o que foi afastado por exceção, e sob qual configuração a decisão foi tomada. O registro bruto fica disponível junto.
- Escolher entre três motores intercambiáveis. Só as regras sobre texto, só os conceitos, ou ambos. Permite medir quanto cada camada contribui e diagnosticar de onde veio uma decisão, se foi a regra sobre o texto ou a norma sobre conceitos.
- Gravar uma trilha em JSONL. Uma linha por processamento, com o princípio, a norma e a configuração que governou cada decisão. Útil para agregar decisões tomadas sob as mesmas regras e responder, por exemplo, quantas vezes cada princípio foi violado ou quais normas mais dispararam.
- Registro da sessão de auditoria. O programa registra quais registros foram abertos, em que ordem e se já haviam sido vistos antes; em que ordem as camadas de detalhe foram expandidas; o tempo de parede e o tempo efetivamente visível de cada registro e da sessão; os filtros aplicados, as marcações de discordância e as tentativas de entrada.
- Avaliar texto. Verificar como o guardrail reage a um trecho isolado, de entrada ou de saída, contra conjuntos de regras distintos, sem chamar nenhum modelo. Cada regra declara em que estágio ela se aplica, e uma regra de entrada nunca é aplicada a uma resposta, e vice-versa. O código de saída indica se houve intervenção, o que permite usá-lo dentro de outros scripts.
- Avaliar datasets. Medir o desempenho do guardrail contra conjuntos rotulados, com erro-padrão para distinguir ganho real de flutuação e metade dos casos reservada para que o número não seja inflado. As divergências saem listadas uma a uma, com o texto e as regras acionadas.
- Demo. Ver o pipeline inteiro funcionando sem precisar instalar nenhum modelo: 7 prompts fixos com respostas simuladas, que percorrem uma liberação, um bloqueio e uma reescrita. Útil para experimentar o sistema antes de configurar um modelo de linguagem.
- Conversar. Falar com um modelo real ou simulado pela tela web, com histórico de conversas e, a cada intervenção, uma faixa dizendo o que foi feito, em que estágio e por qual regra.
- Portabilidade para diferentes modelos de linguagem. O núcleo é importável e não tem dependência externa, o que permite acrescentar a verificação a outro sistema existente.
- Adicionar, remover e editar critérios sem mexer em código. O critério ético vive em arquivos versionados, que o programa aceita por parâmetro junto com a escolha do motor, facilitando a expansão das normas e adaptação do guardrail a outros domínios.

1.3 Usuários primordialmente contemplados

O programa foi concebido para pesquisadores de Interação Humano-IA que precisam de um guardrail que produza registros das decisões tomadas, para estudar como pessoas os leem; e é para esse público que existe a divisão tune/holdout, o registro da sessão de auditoria com tempo e ordem de expansão, e a instrumentação de medição. O projeto também foi pensado para auditores de ética, conformidade ou governança em sistemas de IA sem formação técnica em computação, que precisam julgar se uma decisão do modelo foi adequada, de acordo com as expectativas da organização na qual este se insere. Para esse público, o programa oferece uma tela de auditoria em três camadas de profundidade, com a explicação de cada decisão em linguagem comum e a proveniência apresentada na própria interface, sem exigir que o auditor abra arquivos ou leia código.

1.4 A natureza do programa

Quanto à natureza, trata-se de uma prova de conceito funcional e de uma ferramenta utilitária, que corresponde aos itens #1 e #2 do roteiro de pesquisa de um projeto de mestrado sobre como embutir princípios éticos em agentes baseados em FM. O núcleo é estável e testado, mas segue em desenvolvimento pelo autor. No momento, o guardrail se restringe a Foundation Models, mas será extensível para agentes, no futuro. Pretende-se também ampliar a cobertura normativa, o suporte ao português e algumas capacidades do motor que permanecem incompletas.

2 Visão de Projeto

2.1 Cenário Positivo 1 (i.e. cenário que dá certo)

Marina coordena o comitê de ética de uma seguradora que acaba de ligar um assistente interno para a equipe de atendimento. Para iniciar a revisão, ela entra na tela de auditoria com a sua senha e filtra a lista pelas conversas da semana, descendo pelos distintivos de gravidade até parar num vermelho (DENY). Marina expande a segunda camada do registro e vê a regra que decidiu, o princípio que a justifica, e o trecho exato do pedido que a acionou, com a posição no texto. No entanto, ela discorda da decisão do modelo, acreditando que um DENY não seria a resposta mais apropriada. Ela clica em “Isso deveria ser diferente”, escreve o motivo no campo aberto e registra. O programa registra a objeção dela junto com a decisão e regra que ela contesta, além de registrá-la na própria sessão de auditoria. Posteriormente, quem responde pela política pode ler as objeções de Marina no arquivo de marcações e decidir se altera as regras ou não.

2.2 Cenário Positivo 2

Na seguradora de saúde onde Marina trabalha, a equipe de atendimento usa um assistente interno para consultar e resumir informações de beneficiários. Uma atendente pede o resumo do histórico de um paciente para preparar uma ligação. O modelo responde normalmente, mas inclui na resposta o CPF e o telefone que constavam do cadastro, que são informações confidenciais do beneficiário. O guardrail detecta essas informações sensíveis as redata, mas entrega o restante da resposta. A atendente recebe o histórico completo e faz a ligação, respeitando a privacidade e confidencialidade do beneficiário.

Durante a revisão desse caso, Marina consegue ver no registro normalmente qual regra decidiu, o princípio que a justifica, e as posições exatas de onde os dados foram removidos. No entanto, os valores que foram redatados ficam ocultos até mesmo para ela.

2.3 Cenário Negativo 1 (i.e. cenário que expõe uma limitação conhecida e esperada do programa)

Renata redige a política de contratação da empresa e pede ajuda ao assistente com um trecho: “We should avoid age bias when designing the hiring funnel., mas a requisição volta bloqueada por ferir o princípio de equidade. As provas são “bias” e “designing”, porque a norma exige o par viés + construção para disparar e a frase tem os dois. Só que, no caso de Renata, a proposta é evitar viés ao projetar e o guardrail não consegue distinguir

falar sobre viés de construir com viés. Renata reescreve a frase até passar e, quando tira a palavra “designing”, a mensagem passa. Ela aprendeu a contornar a norma, e na próxima vez que o guardrail a bloquear com razão, ela fará o mesmo.

As normas do RelAIEO são coocorrência de conceitos, sem polaridade. Uma norma só sabe que dois conceitos apareceram juntos, mas não diferencia intenção.

2.4 Cenário Negativo 2

Numa consultoria de tecnologia, um gerente pede ao assistente interno que o ajude a especificar uma ferramenta para espionar seus funcionários. Escreve: “criar um mecanismo que monitore os funcionários da empresa”, e o pedido é bloqueado (DENY). A camada de conceitos reconheceu vigilância e a intenção de construir, e a norma escrita sobre esse casamento negou a requisição. Ele reformula a frase para: “crie um sistema que vigie os funcionários da empresa” e a mensagem passa com sucesso (ALLOW). O modelo responde, e a conversa segue normalmente.

Embora seja o mesmo pedido semanticamente, existe diferença lexical entre “monitore”, que foi alcançado por uma expressão do vocabulário em inglês (por coincidência de radical), enquanto “vigie” não está registrado na lista de conceitos, que só guarda “vigiar” e “vigilância”. O turno é registrado erroneamente como liberado.

Este é o limite esperado de um verificador simbólico. Ele decide só sobre o que foi escrito de antemão e não improvisa julgamento. Para estender a sua cobertura, é necessário ampliar o léxico por um arquivo de configuração.

3 Documentação Técnica do Projeto

3.1 Requisitos funcionais

- RF01**– O sistema deve avaliar os estágios de entrada e de saída separadamente, aplicando a cada um as regras e normas pertinentes.
- RF02**– O sistema deve avaliar um conteúdo textual e produzir um veredito entre ALLOW, FLAG, REWRITE ou DENY, acompanhado das evidências que o motivaram e de uma razão em texto.
- RF03**– O sistema deve avaliar primeiro as constraints rígidas, que sempre resultam em DENY e não admitem exceções.
- RF04**– O sistema deve regras deônticas associadas a princípios éticos, com efeitos graduados e exceções opcionais, incluindo reescrita por template e redação de PII por span.
- RF05**– O sistema deve resolver o conflito entre engines pela decisão mais restritiva, de modo que acrescentar um motor nunca reduza a intervenção.
- RF06**– O sistema deve intervir sem descartar a resposta, apagando identificadores pessoais no texto ou substituindo o conteúdo por formulação declarada em template, conforme o efeito da regra que disparou.
- RF07**– O sistema deve registrar trilha de auditoria completa em JSONL, com regras e normas disparadas, texto casado com offsets, caminhos de inferência e o que foi suprimido por exceção.
- RF08**– O sistema deve oferecer uma tela de auditoria com autenticação, que apresente cada decisão em camadas de profundidade crescente.
- RF09**– O sistema deve carregar as políticas, ontologias e normas de arquivos externos, de modo que possam ser editadas, acrescentadas ou removidas sem alteração de código.
- RF10**– O sistema deve permitir que o auditor registre discordância de uma decisão, sem que isso altere a política, guardando a objeção junto ao registro que a motivou.
- RNF11**– O sistema deve garantir que a falha de uma engine resulte sempre em negação, nunca em liberação (fail-closed).

3.2 Requisitos não funcionais

- RNF01**– Auditabilidade. O sistema deve ser auditável e ser capaz de explicar toda decisão a partir dos artefatos que a produziram.
- RNF02**– Usabilidade. O sistema deve apresentar o registro de decisão de modo compreensível para auditores sem conhecimento de computação.
- RNF03**– Confiabilidade. O sistema deve produzir um veredito de bloqueio diante de qualquer falha na avaliação, nunca liberar conteúdo cuja verificação não tenha sido concluída.
- RNF04**– Determinismo. O sistema deve garantir que a mesma entrada, sob a mesma configuração, produza sempre o mesmo veredito.
- RNF05**– Manutenibilidade. O sistema deve permitir que os critérios éticos sejam alterados por edição de arquivos, sem recompilação nem alteração de código.
- RNF06**– Privacidade. O sistema não deve reter o conteúdo bloqueado nem os dados redigidos em nenhum arquivo, incluindo a trilha de auditoria.
- RNF07**– Rastreabilidade. O sistema deve garantir que todo registro de decisão contenha a versão declarada e o digest de cada arquivo de configuração que a governou, permitindo a comparação entre registros.
- RNF08**– Usabilidade. O sistema deve utilizar uma interface que atenda ao nível AA da WCAG 2.1 quanto ao contraste entre texto e fundo, em ambos os temas.
- RNF09**– Portabilidade. O núcleo depende apenas da biblioteca padrão do Python 3.10 ou superior, sem dependências externas obrigatórias.
- RNF10**– Manutenibilidade. O sistema deve permitir que novas engines, tipos de condição e camadas de verificação sejam acrescentados por contratos declarados, sem alteração do motor de composição nem das camadas existentes.

3.3 Arquitetura e organização do software

3.3.1 A arquitetura

A arquitetura segue o padrão de multi-model guardrails (Liu et al., 2025). O **GuardedAgent**, que cerca o modelo, faz uma verificação ética sobre o texto de entrada e de saída, cuja resposta decide se a mensagem efetivamente chega até o modelo de linguagem ou não. Essa arquitetura delega o julgamento normativo às engines simbólicas, reservando ao LLM somente a geração de conteúdo. Essa separação é o que torna a decisão ética inspecionável e confiável. Como o critério não fica embutido no modelo, que é estocástico e opaco por natureza, as engines externas permitem que o veredito seja traçado, justificado, contestado e calibrado.

O contrato central é a interface **PolicyEngine**, cujo método `evaluate(ActionContext)` devolve um veredito. Ela é implementada por quatro classes. **RuleBasedEngine**, **KnowledgeGraphEngine** e **CompositeEngine** compõem o guardrail. **LLMJudgeEngine** é um protótipo, experimental, mantido fora da composição como voto soft e não auditável, que enxerga apenas a política e nenhuma norma da ontologia. Ele é somente linha de comparação na avaliação, não faz parte do guardrail entregue.

3.3.2 O fluxo de uma verificação

O fluxo tem dois pontos de verificação. Toda entrada ou saída é avaliada por uma engine ao entrar e ao sair do LLM. A intervenção da engine permite quatro vereditos possíveis, ordenados por restritividade: i) **ALLOW**; ii) **FLAG**; iii) **REWRITE**; e iv) **DENY**; além de quatro severidades (**low**, **medium**, **high** e **critical**). Quando várias regras disparam, ou vários motores opinam, vale a decisão mais restritiva. Os vereditos de entrada e de saída permanecem gravados no log de auditoria, com os princípios que os determinaram.

O mesmo caminho é percorrido duas vezes por turno, uma vez sobre a entrada do usuário e outra sobre a resposta do modelo, e as regras que se aplicam são diferentes, de acordo com o estágio. A **RuleBasedEngine** percorre as constraints (regras rígidas que não admitem exceção), e depois as regras da política (que admitem

exceção), produzindo duas listas. A primeira são os casamentos, que são as regras que dispararam sobre aquele texto. A segunda são as supressões, que são as que dispararam mas foram rebaixadas por uma exceção.

Em seguida, a **KnowledgeGraphEngine** avalia as normas sobre os conceitos ativados na ontologia e produz as mesmas duas listas: os casamentos das normas que dispararam e as supressões, que aqui vêm dos conceitos de exceção declarados pela norma e da guarda de frame, mecanismo descrito na seção 5.3. As duas engines recebem o mesmo texto e trabalham de forma independente, nenhuma consultando o resultado da outra. Embora rodem em sequência, a decisão final não depende da ordem em que rodaram.

A **CompositeEngine** é quem chama as duas e monta um veredito novo com o que elas retornaram. Vence a decisão a mais restritiva entre elas. Os casamentos e as supressões são concatenados na ordem em que os motores rodaram, sem reordenação, e é essa ordem que a mensagem de recusa reproduz ao listar os motivos para o usuário. É ela também que trata as falhas, por exemplo, se uma engine levanta exceção, a exceção vira um **DENY** marcado como erro de sistema, e a outra roda mesmo assim.

Não havendo bloqueio, o conteúdo (seja ele original ou reescrito) segue para o modelo de linguagem. A resposta gerada passa pela mesma verificação, agora no estágio de saída. Se também aqui houver bloqueio, a resposta é retida e o usuário recebe a mensagem correspondente. Caso contrário, recebe a resposta liberada ou sua versão reescrita. Havendo um logger configurado, todo o percurso é gravado pelo **AuditLogger**.

3.3.3 Os quatro vereditos

ALLOW — deixar passar Todo veredito começa em **ALLOW**, a menos que alguma regra da política ou alguma norma da ontologia se aplique ao texto. Ele representa uma lista de casamentos vazia, e por isso não pode ser declarado como efeito de disparo de uma regra nem de uma norma. No entanto, pode ser declarado como efeito de supressão. Quando uma regra é isentada por exceção, ela pode declarar **ALLOW** como sucessora.

O texto segue intacto até o modelo, e nada aparece para o usuário. O turno vai para a trilha com a configuração que o governou e conta como não-intervenção na avaliação.

FLAG — anotar sem agir O **FLAG** existe para o caso limiar, quando um texto não é inofensivo, mas a severidade dele não justifica bloqueio. A mensagem segue intacta, e a marca serve apenas para sinalizar ao auditor.

O exemplo é o *ethics washing*, uma acusação sobre a intenção institucional de quem escreveu. Um guardrail simbólico não tem como sustentar essa acusação, e barrar o texto por causa dela seria desproporcional, então a resposta fica marcada para que um humano a revise. Qualquer regra ou norma pode declarar **FLAG**, e uma exceção pode rebaixar uma regra mais severa até ele.

A trilha grava o veredito e o trecho da mensagem, com princípio, justificativa e evidência., **FLAG** não conta como intervenção na avaliação e entra nas métricas como não-detectado.

REWRITE — modifica partes do texto O **REWRITE** é a única decisão que modifica o texto. Ele atua sobre a pergunta que o sistema poderia entender como ofensiva, mas que vem num enquadramento legítimo, de ensino ou de defesa: “how to hack an account” é recusado; “how to hack an account, for a defensive security course” é rebaixado, e o sistema permanece em alerta e sanitiza o conteúdo sensível. Somente o motor de regras pode declarar um **REWRITE**, porque reescrever exige saber exatamente onde intervir no texto, e essa informação só existe na regra, em dois campos.

O primeiro é **redact**, que apaga trechos nas posições que o gatilho já encontrou. Como duas regras podem casar recortes diferentes da mesma região, uma pegando só o número e outra o rótulo junto, os recortes são fundidos antes da substituição, e cada região vira **[REDACTED:<regra>]** com o nome da regra de maior severidade entre as que a produziram. Para o **redact**, o texto original não é guardado na trilha de auditoria.

O segundo é **rewrite_template**, que substitui ou complementa o texto por um template escrito para aquela situação específica. Se aplica só a da regra de maior severidade e o marcador **{content}** decide como ele se combina com o texto original. Com o marcador, o texto original entra no lugar dele. Sem o marcador, o template substitui o texto inteiro, e o original não aparece. Se não houver redação antes, o que entra no marcador é a versão já redigida. Se o **REWRITE** veio de um **rewrite_template**, a trilha de auditoria mantém o texto original, para que o auditor compare o antes e o depois e esse veredito passa a contar como intervenção na avaliação.

DENY — não deixar passar O **DENY** bloqueia o conteúdo por completo. Um **DENY** pode vir das constraints, das regras e normas, e das falhas do sistema. Em casos de falha, quando uma engine levanta exceção e não

pode concluir a verificação, o erro vira **DENY** marcado como erro de sistema. Essa mesma proteção envolve a avaliação inteira, e uma falha fora das engines também resulta em bloqueio.

Se o **DENY** ocorre na entrada, o usuário recebe uma mensagem de recusa montada a partir das regras que dispararam, com identificador, princípio e justificativa de cada uma, e o turno inteiro é omitido do histórico dos turnos seguintes. O registro vai integral para a trilha, incluindo a mensagem bloqueada.

Se ocorrer na saída, a resposta é retida e não é guardada nem na trilha. A decisão é registrada normalmente, exceto pelo texto reprovado. Quando o **DENY** vem de uma falha na chamada do modelo, o turno encerra com estado **system_error** e uma mensagem avisa que o pedido foi negado por precaução. Nesses casos, conta como intervenção na avaliação.

3.3.4 Os agregados de dados

Os dados do sistema se organizam em três agregados, cada um correspondendo a uma parte do julgamento.

A **Policy** reúne e avalia duas listas de **Rule**, começando pelas constraints, que não admitem exceção; depois as regras, que admitem.

A Ontologia reúne três tipos de elemento. Os i) **Concept**, que são as ideias nomeadas na ontologia, sobre as quais as normas são escritas. Cada um agrega suas ii) **Lexicalization**, que são as expressões gatilho do texto capazes de ativar cada conceito. As iii) **Relation** ligam conceitos entre si através de uma a relação **is_a**, que faz a ativação propagar de um conceito específico para outros conceitos mais gerais na ontologia. Sobre esse grafo ficam as **Norm**, que declaram quais conceitos precisam estar ativos e decidem qual efeito aplicar e por qual princípio.

O **Verdict** é a decisão do modelo, que carrega o estágio em que foi tomada, o motor que a produziu, o conteúdo reescrito quando houve, um resumo em texto e um sinalizador de erro de sistema. Nele ficam também duas listas, o **RuleMatch** e o **SuppressedMatch**.

Os **RuleMatch** registram os gatilhos dos disparos. Cada um traz o identificador da regra, o princípio que a justifica, a deontologia (proibição ou obrigação), a severidade, o efeito, a justificativa e se é rígida ou não. Também traz sua **Evidence**, que guarda uma descrição para cada casamento, o trecho literal de texto e sua posição em caracteres no original.

Os **SuppressedMatch** registram as exceções, com a regra ou norma suprimida, a razão, a evidência e o efeito que passou a valer no lugar. Essa supressão pode vir por exceções de uma regra, por conceitos **unless** de uma norma, pela guarda de frame.

A parte desses agregados, o acesso a modelos de linguagem é abstraído pela interface **LLMClient**. O modelo **Ollama** pode ser utilizado localmente ou na sua versão em nuvem, caso haja uma chave de API. A **MockLLM** opera offline, e serve para ser usada tanto nos testes quanto quando nenhum modelo está disponível.

3.3.5 Os motores

Motor de regras A responsabilidade do motor de regras é ler as políticas, localizadas em um arquivo JSON com três constraints e nove regras. Cada regra declara uma condição, montada a partir de sete tipos combináveis, em três grupos. **Keyword** e **Regex** casam texto (“contém esta palavra”, “casa este padrão”). **Any**, **All** e **Not** compõem outras condições (“qualquer uma destas”, “todas estas”, “esta não”), e é combinando os dois grupos que uma regra expressa critérios elaborados.

Concept e **Refusal** formam um terceiro grupo, porque ligam a política às demais camadas em vez de casar texto. A **Concept** permite que uma regra pergunte se um conceito da ontologia está ativo, com a inferência incluída; por padrão, ela considera tanto as ativações diretas quanto as herdadas. A **Refusal** é usada no bloco de exceções e desliga a regra quando o que a disparou está dentro de uma recusa do modelo, e vale só para respostas. As duas estão implementadas, mas a política atual ainda não as usa, por limitação do corpus de regras e constraints.

Montada a condição, a avaliação verifica se ela casa, e a regra dispara. Se a regra declara **exceptions** e elas também casam, ela é rebaixada para onde o campo **suppressed_effect** mandar. O rebaixamento vira uma entrada em **suppressed**, com a razão e o efeito sucessor, para que o auditor distinga um pedido liberado de um rebaixado.

Quando uma regra **redact** dispara, o trecho que ela casou some, inclusive da trilha. O registro guarda a posição, mas apaga o valor.

Motor de grafo de conhecimento Um léxico de 54 expressões ancora trechos do texto em conceitos de duas taxonomias. A RelAIEO [citar], vendorizada de terceiros, traz 154 conceitos, dos quais 8 têm ancoragem, e 6 normas operam sobre eles. A taxonomia de dano, escrita para este trabalho, traz 12 conceitos, todos ancorados, com 12 normas próprias, e se pendura na RelAIEO por cinco relações `is_a`. Somadas, são 167 conceitos e 18 normas.

Cada conceito lexicalizado procura suas expressões no texto, e o que casa vira evidência com trecho e posição. A ativação então propaga pela hierarquia, do caso particular para o geral, registrando o caminho na descrição. É o que faz um texto sobre vigilância dirigida acionar uma norma escrita sobre vigilância.

Cada norma declara os conceitos que precisam estar ativos (**when**), os que a afastam (**unless**), o estágio em que vale, e um efeito que só pode ser **DENY** ou **FLAG**. Quando dispara, sua justificativa recebe as provocações dos conceitos envolvidos, que são perguntas dirigidas ao auditor humano, como “Quem é a pessoa vigiada nesta resposta, e ela teria como saber que foi?”.

A camada de frames Existe para resolver um problema do estágio de saída, no qual uma resposta que recusa um pedido perigoso carrega o mesmo vocabulário da resposta que o cumpre. Sem tratar isso, o guardrail bloquearia também as recusas corretas do modelo. O mecanismo é o do algoritmo ConText [citar], que utiliza gatilhos com direção e alcance em tokens para marcar trechos do texto como recusa. A guarda isenta a norma quando todos os casamentos que a ativaram caem dentro de um trecho de recusa.

O composto O motor padrão é o híbrido, com regras e grafo rodando juntas sobre o mesmo texto. As regras disparadas e as isentadas de ambos os motores entram no mesmo veredito. Se um motor levantar exceção, ele vira um veredito **DENY** marcado como erro de sistema, que é a falha fechada prevista.

3.3.6 A trilha de auditoria

Cada mensagem grava uma linha JSON. Ela identifica o evento com um UUID, um carimbo de tempo UTC e o par conversa/turno; registra o estado, o motor que decidiu, a origem e a proveniência da configuração; guarda o texto de entrada, os vereditos de entrada e de saída, a mensagem entregue, os textos reescritos e a resposta bruta do modelo quando ela foi retida; e anota qual modelo respondeu, ou o erro dele (se houver).

A proveniência serve para saber qual configuração de fato governou cada decisão. Para cada arquivo de configuração usado naquela decisão, ela grava papel, versão declarada, caminho e `sha256` do conteúdo, e sobre esse conjunto calcula um `config_id`, por uma receita nomeada (`config-id/v1`). O programa avisa, na primeira gravação de cada processo, onde a trilha vive; e se a escrita falhar, reporta o erro e continua sem registrar aquele evento.

3.3.7 A interface de auditoria

A interface de auditoria é protegida por uma senha definida no momento da instalação. Essa senha é apenas separação de papéis, porque ela vale apenas para a máquina na qual o programa foi instalado e é removida junto com o programa. A tela de auditoria permite visualizar os vereditos em três camadas:

A camada 1 fala em linguagem comum, e contém a mensagem do usuário, o veredito e por quê, com o princípio e o número de normas que se aplicaram.

A camada 2 traz os vereditos de entrada e de saída, as regras que foram disparadas com identificador, efeito, princípio, severidade e justificativa, e cada evidência com o trecho e sua posição. Quando houve reescrita, mostra o antes e o depois.

A camada 3 traz é a própria proveniência e diz qual motor decidiu, quais arquivos governaram a decisão, com versão declarada e digest, o `config_id` e qual modelo respondeu.

Por fim, vem o registro bruto da conversa. Quando a resposta bloqueada não está no registro, a tela diz que ela nunca é guardada. Quando houve redação de trecho, diz isso no lugar exato onde ele estaria.

Caso o auditor discorde de um veredito, é possível marcar essa discordância e anexar uma mensagem. A marcação vai para uma trilha própria, separada da trilha do agente, com o registro julgado, a regra, a nota do auditor e a configuração vigente na decisão. A marcação é registrada como intenção e não altera nada, porque a edição das regras é estritamente manual.

O auditor também é auditado. Uma tarja não-dispensável informa que a sessão está sendo registrada, e o auditor pode ler os próprios eventos, leitura que também é registrada.

3.3.8 Configuração

O critério ético vive fora do programa, em nove arquivos versionados. Existe a política, as duas taxonomias em Turtle, os dois léxicos de ancoragem, os dois arquivos de normas e os gatilhos de frames. Um parâmetro de linha de comando seleciona o caminho de cada um, o motor (**rule**, **kg** ou **hybrid**) e o destino da trilha. Acrescentar, remover e editar critério é editar esses arquivos. programa só precisa ser tocado se o domínio exigir um princípio fora do vocabulário fechado de oito que ele verifica na carga.

3.3.9 As frentes de uso

Tanto a interface quando o prompt permitem alguns comandos. São eles:

Check: avalia um texto isolado. Ele não chama o modelo de linguagem e não gera resposta nenhuma. Serve para perguntar o que o guardrail decide sobre aquele texto, e devolve o veredito com a explicação e as evidências. Como a verificação de entrada e a de saída seguem regras diferentes, o estágio é declarado na chamada.

Process: roda o pipeline completo. Verifica a entrada, consulta o modelo se ela passar, verifica a resposta, e entrega o que sobreviveu às duas verificações. É o comando que exercita o sistema como ele funciona em uso real.

Demo: roda sete caso roteirizado, sempre offline, com o MockLLM no lugar do modelo. Mostra o percurso inteiro sem exigir modelo, rede ou configuração, e serve para ver o que o sistema faz antes de preparar qualquer coisa.

Eval: roda o arranjo de avaliação sobre benchmarks, que podem ser alterados. Compara o veredito do sistema com o esperado e reporta acurácia, precisão, recall, F1 e a matriz de confusão. Diferente dos três anteriores, ele nunca escreve na trilha de auditoria, para que medir o instrumento não contamine o que o instrumento registra.

Modelo de linguagem: local, por um servidor Ollama em localhost, ou em nuvem, quando há chave de API. Qual dos dois respondeu fica registrado. Se nenhum estiver disponível, o programa não falha em silêncio nem finge: cai para uma simulação e grava a razão da queda e o modelo que havia sido pedido, e toda interface que mostra conteúdo mostra também a frase que diz o que o produziu.

3.4 Sobre o código

O programa é escrito em Python 3.10 ou superior e distribuído sob a GNU GPL v3 ou posterior. O núcleo usa apenas a biblioteca padrão. As dependências opcionais `ollama` e `python-dotenv`, agrupadas no extra `llm`, só entram para conectar um modelo real, que é opcional em todas as tarefas. Na falta de um modelo, o sistema recorre ao MockLLM, que roda offline. Um segundo extra, `dev`, traz o ferramental de teste.

Três construções da linguagem sustentam os contratos. As dataclasses declaram todos os campos de um tipo em um só lugar. Os enums fecham as listas de opções, de modo que escrever qualquer coisa fora delas é erro detectado imediatamente, e são quatro decisões, quatro severidades e dois estágios. O vocabulário de oito princípios obedece ao mesmo fechamento, verificado no carregamento. As classes abstratas `Condition`, `PolicyEngine` e `LLMClient` definem o que uma condição, uma engine e um cliente de LLM precisam oferecer para serem aceitos.

O pacote `ethical_agent` se organiza em módulos de responsabilidade única, agrupados pelas camadas do sistema. Os tipos comuns ficam em `types.py`.

A camada de regras reúne quatro módulos.

- `conditions.py`, a árvore de condições e o registro extensível
- `policy.py`, os modelos `Rule` e `Policy`, com carregador e validação
- `engine.py`, a interface `PolicyEngine`, a `RuleBasedEngine` e a `CompositeEngine`
- `frames.py`, a camada de quadros de recusa, que entra no sistema pela condição `refusal` e pela guarda `unless_frame`

A camada de ontologia reúne outros três.

- `ontology.py`, que define `Concept`, `Lexicalization`, `Relation` e `Norm`, e a ativação no grafo

- `relaieo.py`, o leitor Turtle sem dependências, que carrega as duas taxonomias em união com a fonte declarada em cada conceito
- `kg_engine.py`, a `KnowledgeGraphEngine`

Os demais módulos sustentam execução, verificação e distribuição.

- `agent.py`, o pipeline `GuardedAgent`
- `llm.py`, a interface `LLMClient` com `MockLLM` e `OllamaClient`
- `audit.py`, o logger JSONL
- `provenance.py`, os artefatos de configuração e o `config_id`
- `evaluate.py`, o harness de avaliação
- `llm_judge.py`, a engine experimental
- `demo.py`, os prompts do demo offline
- `senha_auditoria.py`, a senha da tela de auditoria
- `__main__.py`, a CLI

O subpacote `webui/` traz a interface web local. Um conjunto de módulos cobre a instalação e a desinstalação, acionados pelos scripts de raiz `wizard_gui.py`, `uninstall_gui.py` e `audit_tools.py`.

3.4.1 Resultados da avaliação

A avaliação conta com três datasets externos, que foram divididos ao meio antes de qualquer medição. Essa decisão foi tomada porque é necessário ver os erros do guardrail para melhorá-lo. Para cada avaliação, é preciso identificar quais casos divergiram do rótulo, descobrir que a palavra que ativaria a regra falta no léxico, acrescentá-la e rodar de novo para que ele acerte na próxima vez. Cada caso corrigido assim deixa de testar o sistema, porque ele passa a acertar aquele exemplo específico, mas não a generalizar a regra.

A solução foi dividir cada dataset (exceto o padrão) em dois, com um **Tune** e um **Holdout** para cada. A ideia é testar as correções na versão **Tune**. Rodar nele as avaliações, ver em quais casos o sistema acertou e errou e ajustar a regra ou o léxico de acordo. Depois de algumas correções, a versão **Tune** poderá ser comparado com a versão **Holdout**, que é a versão sem as correções, para quantificar a melhora na performance do sistema.

A divisão dos datasets foi feita calculando o hash do identificador de cada conjunto, cujo resultado foi utilizado para decidir a metade na qual cada caso foi designada. Essa maneira de separar os datasets foi escolhida por ser determinística, portanto, reproduzível.

No entanto, não há como garantir que as duas metades tenham a mesma proporção de casos que devem ser bloqueados, mas a acurácia e o F1 dependem dessa proporção. Essa diferença afeta a acurácia e o F1. Comparar acurácia entre duas metades com composições diferentes misturaria o desempenho do sistema com a mistura do conjunto e contaminaria o resultado. Para isso, esse estudo usa recall para contar quantos dos casos que deveriam ser bloqueados o sistema pegou, e ignora todo o resto. Se uma metade tem mais casos a bloquear que a outra, o denominador acompanha, e a fração continua medindo a mesma coisa.

Essa também é a razão pela qual o dataset curado ficou de fora da separação. Os casos desse conjunto foram escritos pela mesma pessoa que escreveu as regras, e foram escritos justamente para testar essas regras. Logo, dividir esse conjunto não produziria um holdout de verdade. Por isso, o programa reporta esse conjunto sempre inteiro, e sempre em separado dos outros.

3.4.2 Controle de qualidade

O controle de qualidade é automatizado com `pytest`, sem plugins: `python -m pytest` na raiz. A suíte tem 1 306 testes em 66 arquivos, cobrindo as engines, a ontologia, o pipeline do agente e o carregamento e validação das configurações, com testes de unidade por componente e uma varredura exaustiva do reticulado de decisões.

Para cada caso da grade, a decisão composta é confrontada com `most_restrictive` das decisões de camada. Duas travas de linha de base em `tests/test_eval.py` garantem que a engine híbrida nunca fique pior que a de

regras em acurácia de decisão e recall, e se mantenha sempre acima do limiar estabelecido de 0,9 em acurácia binária, acurácia de decisão e recall. Na execução de referência (commit 3300649, feita em 2026-08-08), a suíte completa passa 1 305 testes com 1 pulado, dos 1 306 coletados, em cerca de 210 s.

Tabela 1: Contribuição da camada de conceitos ao recall, por conjunto e metade

Conjunto	Metade	Casos	Prevalência de positivos	Recall rule	Recall hybrid	Δ recall
Dataset curado	Full	72	68,1%	$0,714 \pm 0,065$	$0,980 \pm 0,020$	+0,265
BeaverTails	Tune	117	55,6%	$0,046 \pm 0,026$	$0,523 \pm 0,062$	+0,477
BeaverTails	Holdout	103	53,4%	$0,055 \pm 0,031$	$0,382 \pm 0,066$	+0,327
Injections	Tune	323	41,2%	$0,015 \pm 0,011$	$0,015 \pm 0,011$	0
Injections	Holdout	339	38,3%	$0,054 \pm 0,020$	$0,062 \pm 0,021$	+0,008

recall com erro-padrão binomial · config_id (hybrid, 8 artefatos) 7a41bfbfd69 · medido em 2026-08-08 · HEAD 3300649

A Tabela 1 reúne o desempenho dos dois motores extremos em cada conjunto e cada metade. A coluna Δ é a contribuição da camada de conceitos, medida como a diferença de recall entre a camada de regras isolada e a composição das duas.

A leitura vertical da coluna Δ mostra que, no conjunto curado, a camada de conceitos acrescenta 0,265, no BeaverTails acrescenta 0,477 e 0,327, e no deepset não acrescenta nada, com 0 no tune e 0,008 no holdout. A camada que decide varia de acordo com o conjunto. A camada de conceitos, por exemplo, alcança formulações que nenhuma regra prevê, porque uma norma escrita sobre um conceito vale para todas as expressões ancoradas nele, mas ela só alcança o que a ontologia modela. No deepset, cujo assunto é injeção de prompt, não há conceito correspondente para ativar, e a camada fica inerte.

A queda entre as duas metades do BeaverTails, de 0,523 para 0,382, mede generalização. Como a métrica comparada é o recall, essa queda vem do desempenho do sistema diante de casos que não participaram do ajuste, resultando em uma perda que é de aproximadamente um quarto do recall.

As duas linhas do deepset são a outra face da limitação de cobertura, com a camada de regras alcançando 0,015 no tune e 0,054 no holdout. Mesmo assim, nenhuma das duas camadas alcança o material desse dataset.

O conjunto curado aparece com recall 0,980 contra 0,382 do holdout externo. É um resultado esperado, já que o conjunto foi escrito pelo próprio autor para testar as regras.

Tabela 2: Métricas dos três motores sobre o conjunto curado, inteiro

Métrica	rule	kg	hybrid
Casos (N)	72	72	72
Prevalência de positivos	0,681	0,681	0,681
Taxa de intervenção	0,486	0,194	0,681
Acurácia binária	0,806	0,486	0,972
Precisão	1,000	0,929	0,980
Recall ($\pm$ e.p.)	0,714 $\pm$ 0,065	0,265 $\pm$ 0,063	0,980 $\pm$ 0,020
F1	0,833	0,413	0,980
Confusão (TP/FP/FN/TN)	35/0/14/23	13/1/36/22	48/1/1/22
Acurácia de decisão exata	0,806	0,486	0,972

conjunto curado · metade inteiro · config_id (hybrid, 8 artefatos) **7a41bfbffd69** · medido em 2026-08-08 · HEAD 3300649

A Tabela 2 traz as métricas completas dos três motores sobre o conjunto inteiro curado, sem divisão. A Figura demonstra que a camada de regras vence em precisão, com nenhum falso positivo em 72 casos, e a híbrida vence nas demais.

A híbrida acrescenta 13 detecções à camada de regras e paga por elas um único falso positivo, o que leva o recall de 0,714 a 0,980. A taxa de intervenção da híbrida, 0,681, coincide com a prevalência de positivos do conjunto, ou seja, quantos dos casos que exigiam intervenção o sistema efetivamente pegou, e quantos deixou passar. A matriz 48/1/1/22 mostra que ela também intervém quase sempre nos casos certos.

Tabela 3: Métricas dos três motores sobre a metade reservada do BeaverTails

Métrica	rule	kg	hybrid
Casos (N)	103	103	103
Prevalência de positivos	0,534	0,534	0,534
Taxa de intervenção	0,039	0,252	0,272
Acurácia binária	0,485	0,602	0,602
Precisão	0,750	0,769	0,750
Recall ($\pm$ e.p.)	0,055 $\pm$ 0,031	0,364 $\pm$ 0,065	0,382 $\pm$ 0,066
F1	0,102	0,494	0,506
Confusão (TP/FP/FN/TN)	3/1/52/47	20/6/35/42	21/7/34/41
Acurácia de decisão exata	0,466	0,602	0,592

conjunto BeaverTails · metade holdout · config_id (hybrid, 8 artefatos) **7a41bfbffd69** · medido em 2026-08-08 · HEAD 3300649

Ressalva do harness acurácia e F1 não são comparáveis com a outra metade (gap 0,022 > 0,02); entre metades compare recall, que é invariante à mistura.

A Tabela 3 traz as métricas dos três motores sobre a metade reservada do BeaverTails. Aqui a camada de conceitos carrega o peso da decisão. Ela sozinha alcança recall 0,364 contra 0,055 da camada de regras, e vence em precisão e em acurácia de decisão exata.

Comparando as camadas de conceitos e a híbrida, é possível notar que a híbrida detecta um caso a mais, 21 contra 20, e adquire junto um falso positivo a mais, 7 contra 6. Existe um ganho de recall, mas custa precisão, e a acurácia de decisão exata cai de 0,602 para 0,592, porque num caso a composição chega a uma decisão diferente da ideal. É um trade-off esperado entre maior amplitude de detecção e exatidão.

Tabela 4: Métricas dos três motores sobre a metade reservada do deepset/prompt-injections

Métrica	rule	kg	hybrid
Casos (N)	339	339	339
Prevalência de positivos	0,383	0,383	0,383
Taxa de intervenção	0,021	0,006	0,024
Acurácia binária	0,637	0,622	0,640
Precisão	1,000	1,000	1,000
Recall ($\pm$ e.p.)	0,054 $\pm$ 0,020	0,015 $\pm$ 0,011	0,062 $\pm$ 0,021
F1	0,102	0,030	0,116
Confusão (TP/FP/FN/TN)	7/0/123/209	2/0/128/209	8/0/122/209
Acurácia de decisão exata	0,637	0,622	0,640

conjunto deepset/prompt-injections · metade holdout · config_id (hybrid, 8 artefatos) **7a41bfbfd69** · medido em 2026-08-08 · HEAD 3300649

Ressalva do harness acurácia e F1 não são comparáveis com a outra metade (gap 0,028 > 0,02); entre metades compare recall, que é invariante à mistura.

A Tabela 4 traz as métricas dos três motores sobre a metade reservada do deepset. Embora a precisão marque 1,000 nas três colunas, a taxa de intervenção mostra que o conjunto tem 38,3% de casos que deveriam ser bloqueados, mas a camada híbrida intervém em apenas 2,4% deles. Ela acerta os oito casos em que dispara, e deixa passar 122. Esse nível de precisão descreve um motor que age e intervém pouco, que também é um efeito esperado devido às limitações de constraints e regras disponíveis.

É o que justifica a falta de contribuição da camada de conceitos, com 2 detecções em 339 casos contra 7 da camada de regras. O sistema performa particularmente pior nesse dataset porque o léxico e as normas foram escritos sobre danos éticos, e este conjunto trata de injeção de prompt, que fica fora do vocabulário previsto.

4 Considerações finais

Este trabalho entrega uma prova de conceito funcional. O núcleo é estável, testado e mede o próprio desempenho, mas a cobertura normativa está no começo. Hoje o guardrail verifica 8 conceitos ancorados na ontologia vendorizada e 12 na taxonomia de dano escrita para este trabalho, sobre um léxico de 54 expressões. Os resultados deste trabalho refletem a cobertura limitada que esse léxico e essas normas dão atualmente. Na avaliação da camada de conceitos no BeaverTails, porém, o recall vai de 0,046 a 0,523. Esse ganho vem de uma propriedade do desenho, que é uma norma escrita sobre um conceito valer para todas as expressões ancoradas nele. Cada conceito novo com ancoragem estende essa cobertura, e cada expressão acrescentada ao léxico passa a acionar de uma vez todas as normas daquele conceito.

Portanto, o caminho de evolução mais direto é ampliar léxico e normas. A queda de 0,523 para 0,382 entre as metades do BeaverTails mostra o tamanho do que falta ampliar, já que parte das correções feitas contra a metade de ajuste acrescentou expressões específicas em vez de estender conceitos, e o holdout mede o quanto disso não transferiu. A arquitetura acomoda essa ampliação, que acontece por edição de arquivo e não por alteração de código.

Porém, existem duas limitações que não podem ser resolvidas por ampliação. A primeira é o alcance de domínio. No conjunto de injeção de prompt, a camada de conceitos decide sozinha em 1 caso de 339, porque não existe conceito correspondente para ativar. Alcançar esse material exige modelar um domínio novo, e não

acrescentar termos ao existente. A segunda é a polaridade. As normas descrevem coocorrência de conceitos e não distinguem intenção. A frase "We should avoid age bias when designing the hiring funnel" é bloqueada pela norma N-REL-005, com as evidências bias e designing registradas nas posições 20..24 e 30..39, ainda que a proposta seja justamente evitar o viés ao projetar. O cenário negativo 1 narra a consequência dessa limitação para quem a encontra. Nenhuma quantidade de conceitos novos a resolve, porque ela está na forma da norma e não na cobertura.

Essas limitações não invalidam a proposta do trabalho, porque o objetivo não era amplitude de cobertura, e sim auditabilidade. Entregar um guardrail auditável, cujas camadas são desacopladas do Foundation Model e mensuráveis, de forma que gere um registro confiável o suficiente para que alguém conteste uma decisão sem ler código. Essas propriedades já estão implementadas e medidas, e a ampliação da cobertura das normas é um trabalho futuro.

5 Manual de Utilização para Usuários Contemplados

Este manual cobre os dois perfis de usuário contemplados, o auditor e o pesquisador. Ele está organizado por tarefa, porque quase tudo que o sistema faz pode ser feito tanto pela interface quanto pela linha de comando. Há ainda um terceiro caminho, para quem escreve código próprio, que é o uso como biblioteca (Tarefa H).

A interface tem cinco telas, nomeadas na barra de navegação: Conversa, Avaliar texto, Demo, Eval e Auditoria.

5.1 Tarefa A — Instalação

```
Guia de Instruções:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Para [Tarefa A: INSTALAR] faça:
Passo 1: Execute o assistente de instalação.
    > py -3 wizard_gui.py
Passo 2: Percorra as páginas, confirmando cada uma.
Passo 3: Para usar a tela Auditoria, preencha a senha quando o
    assistente pedir. A senha vale somente para a máquina atual.
Passo 4: Ao terminar, o assistente prepara o ambiente Python do projeto
    (a pasta .venv), sobe o servidor e abre a interface no
    navegador sozinho.

>>> Esta tarefa se faz uma vez só. Para abrir a interface nas próximas
    vezes, veja a Tarefa B.

>>> O servidor fica apenas em 127.0.0.1, o endereço da própria
    máquina. Não fica exposto na rede.

>>> A instalação não cria atalho. O que ela deixa pronto é o ambiente
    Python; abrir a interface depois é subir o servidor por ele.

Exceções ou potenciais problemas:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Se [o assistente terminar mas o navegador não abrir]
{
    Então faça: abra http://127.0.0.1:8765 manualmente. O servidor já
        está no ar e o que falhou foi só a abertura automática.
}

Se [você quiser encerrar o servidor que o assistente subiu]
{
    Então faça: use o comando taskkill que o próprio assistente imprime,
        com o número do processo.
    É porque: o assistente sobe o servidor destacado de si mesmo, para
        que ele sobreviva ao fim da instalação. Não há janela de
        terminal para interromper com Ctrl+C.
}

Se [você quiser desinstalar]
{
    Então faça: > py -3 uninstall.py
    É porque: ele encerra o servidor antes de remover o ambiente. Remover
        com o servidor no ar deixaria arquivos presos e a remoção
        pela metade.
}
```

5.2 Tarefa B — Abrir a interface (com o programa já instalado)

```

Guia de Instruções:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Para [Tarefa B: ABRIR A INTERFACE] faça:
Passo 1: Abra o PowerShell e entre na pasta do projeto.
> cd C:\Users\Gabriel\Downloads\Projeto\ai-ethical-agent
Passo 2: Suba o servidor com o Python do ambiente preparado na instalação.
> .venv\Scripts\python.exe -m ethical_agent serve
Passo 3: Abra http://127.0.0.1:8765 no navegador.
Passo 4: Para parar, dê Ctrl+C na mesma janela do PowerShell.

>>> A trilha de auditoria é gravada em logs\audit.jsonl relativo à pasta
     em que o comando roda e subir o servidor de outro lugar escreve a trilha
     em outro lugar.

>>> Alternativa, se preferir ativar o ambiente antes:
> .venv\Scripts\Activate.ps1
> ethical-agent serve
Mesmo resultado, comando mais curto.

>>> Para usar outra porta: serve --port 9000. Em qualquer porta, o
     servidor continua escutando só em 127.0.0.1.

>>> Fechar a aba do navegador não encerra o servidor. Ele é um processo
     à parte, e reabrir o endereço volta para a mesma sessão.

Exceções ou potenciais problemas:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Se [aparecer "a porta 8765 já está em uso"]
{
    Então faça: leia a mensagem antes de agir. Ou um servidor deste
                projeto já está no ar, ou existe outro programa ocupando
                a porta - e sugere --port 8766.
    É porque: se for um servidor deste projeto, basta abrir o navegador
                no endereço; não é preciso subir nada.
}

Se [a interface abrir com uma configuração que você não reconhece]
{
    É porque: o que você está vendo é um servidor antigo, de uma sessão
                anterior, que ficou no ar segurando a porta. Encerre-o e
                suba de novo, ou a tela continuará respondendo pela
                configuração velha.
}

Se [a página abrir mas nenhuma ferramenta responder]
{
    Então faça: confira no terminal se o servidor registrou erro ao
                carregar a política ou a ontologia.
    É porque: a interface é servida mesmo quando a configuração falha ao
                carregar; a falha aparece na primeira chamada, não na
                abertura.
}

```

5.3 Tarefa C — Checar um conteúdo e ler a sua explicação

```

Guia de Instruções:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Para [Tarefa C: CHECAR UM CONTEÚDO E LER A EXPLICAÇÃO] faça:

Pela interface:
Passo 1: Vá à tela Avaliar texto.
Passo 2: Escreva ou cole o conteúdo a examinar e submeta.
Passo 3: Leia o veredito e, abaixo dele, a explicação.

Pela linha de comando:
Passo 1: Execute o comando check com o texto entre aspas.
$ python -m ethical_agent check "Should I use race as a feature in my credit scoring model?"
Passo 2: Leia a decisão e a explicação com as evidências.

Passo comum: a explicação nomeia a regra ou norma que disparou, mostra o
                trecho exato do texto que a fez disparar, o caminho de
                inferência no grafo quando houver, e o que foi suprimido
                por exceção.

>>> Rodar o mesmo texto em rule e em kg e

```

```

comparar é o que isola de qual camada veio uma decisão.
Na linha de comando:
    $ python -m ethical_agent --engine rule check "algum texto"
    $ python -m ethical_agent --engine kg check "algum texto"
Na interface, a engine é a que estiver escolhida no painel de
Configuração.

Exceções ou potenciais problemas:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Se [o conteúdo for considerado sensível, na linha de comando]
{
    É porque: a saída traz a decisão e a explicação, e o código de saída
    é 2. Isso é esperado, sinaliza que o guardrail interveio.
}

Se [o veredito parecer errado]
{
    Então faça: confira o trecho casado antes de concluir. A explicação
    mostra exatamente o que o sistema leu; muitas divergências
    são de leitura. Caso contrário, altere as regras.
}

Se [a explicação citar uma regra que você não conhece]
{
    Então faça: procure o identificador dela no arquivo de política. Todo
    identificador que aparece na explicação existe lá.
}

Se [uma engine falhar internamente]
{
    Então faça: verifique a configuração JSON carregada (política ou
    ontologia malformada).
    É porque: a política fail-closed nega o conteúdo por segurança quando
    uma engine falha. A negação é a resposta correta a uma
    falha.
}

```

5.4 Tarefa D — Conversar com um modelo protegido pelo guardrail

```

Guia de Instruções:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Para [Tarefa D: CONVERSAR COM UM MODELO PROTEGIDO PELO GUARDRAIL] faça:

Pela interface:
Passo 1: Vá à tela Conversa.
Passo 2: Escreva a mensagem e envie.
Passo 3: Leia a resposta e o status: aprovada, bloqueada ou reescrita.

Pela linha de comando:
Passo 1: Execute o comando process com o texto entre aspas.
    $ python -m ethical_agent process "Por que o céu é azul?"
Passo 2: Leia o status e a resposta.

>>> Opções da linha de comando:
--model MODELO    escolhe o modelo Ollama
--mock            ignora o Ollama, usa resposta fixa do MockLLM
--verbose         imprime os vereditos de entrada e de saída
--json            imprime o resultado como dado estruturado

>>> O modelo padrão é llama3.2:3b. Ele pode ser trocado de três formas,
da mais específica à mais geral: a opção --model, a variável
OLLAMA_MODEL no arquivo .env, ou o padrão embutido.

>>> Para usar um modelo Ollama de verdade, configure um arquivo .env na
raiz do projeto:
- Ollama Cloud: defina OLLAMA_API_KEY (chave gerada em
  https://ollama.com/settings/keys); não precisa instalar nada
  localmente.
- Ollama local: instale via https://ollama.com/download, rode
  "ollama serve" e baixe o modelo escolhido.
Em ambos os casos: $ pip install ollama python-dotenv

Exceções ou potenciais problemas:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Se [o Ollama não estiver configurado ou não responder]
{
    É porque: o sistema tenta o modelo real primeiro e cai automaticamente
    para o MockLLM se ele falhar.
}

```

```

}

Se [a resposta vier reescrita e você quiser ver o original]
{
    Então faça: na linha de comando, repita com --verbose. O veredito de
                saída nomeia o que foi reescrito e por qual regra.
}

```

5.5 Tarefa E — Rodar a demonstração

Guia de Instruções:
 //////////////////////////////////////
 Para [Tarefa E: RODAR A DEMONSTRAÇÃO] faça:

Pela interface:
 Passo 1: Vá à tela Demo.
 Passo 2: Clique em "Rodar demo".
 Passo 3: Acompanhe as etapas do pipeline, da entrada à resposta.

Pela linha de comando:
 Passo 1: Execute o comando demo.
 \$ python -m ethical_agent demo
 Passo 2: Acompanhe as mesmas etapas.

>>> A demonstração roda sempre com respostas simuladas, pelos dois caminhos. Nunca chama um modelo de verdade, é o que a torna previsível e reproduzível.

>>> Ela existe para mostrar o pipeline inteiro, o caminho mais curto para ver o que o sistema faz sem precisar conversar com ele.

>>> A demonstração é registrada na trilha de auditoria, no mesmo arquivo das decisões reais. Cada registro dela vem marcado com a origem "demo", que é como se distingue um do outro na leitura (Tarefa G).

Exceções ou potenciais problemas:
 //////////////////////////////////////

Se [você não quiser a demonstração misturada às decisões reais]
 {
 Então faça: rode-a com o servidor subido de outra pasta, ou aponte a trilha para outro arquivo antes de rodar.
 É porque: os registros vão para a mesma trilha, marcados pela origem. A marca permite separá-los na leitura, mas não os mantém em arquivo à parte.
 }

5.6 Tarefa F — Executar a avaliação

Guia de Instruções:
 //////////////////////////////////////
 Para [Tarefa F: EXECUTAR A AVALIAÇÃO] faça:

Pela interface:
 Passo 1: Vá à tela Eval.
 Passo 2: Escolha o dataset, a engine e a extensão do dataset.
 Passo 3: Execute e leia o relatório. Sai uma caixa por avaliação, titulada com o dataset, a engine e a metade.

Pela linha de comando:
 Passo 1: Execute o comando eval (por padrão, a engine híbrida sobre o conjunto curado inteiro).
 \$ python -m ethical_agent eval
 Passo 2: Para trocar a engine ou a metade, use --engine e --half.
 \$ python -m ethical_agent --engine rule eval --half holdout

Passo comum: leia o bloco Divisão, que abre o relatório, ANTES das métricas. Ele diz qual conjunto foi avaliado, qual metade, quantos casos, e a proporção entre casos que deveriam ser bloqueados e casos que não deveriam. Só depois leia acurácia, precisão, recall, F1 e matriz de confusão.

>>> Por que o bloco Divisão vem primeiro: a acurácia depende da mistura do conjunto. Um conjunto com poucos casos que deveriam ser bloqueados produz acurácia alta mesmo num sistema que quase não

```

    bloqueia. Sem a proporção, o número engana.

>>> A engine escolhida vale só para esta execução, e não muda a engine
da conversa. "Comparar as três" roda Rule, KG e Hybrid sobre os
mesmos casos e devolve uma caixa para cada, o que mostra quanto
cada camada contribui.

>>> A extensão do dataset decide qual metade é avaliada:

    Inteiro    - o conjunto todo, sem separar ajuste de reporte.
    Tune       - a metade cujos erros se pode ler. Quando o sistema deixa
passar um caso que deveria bloquear, é aqui que você vê
qual foi, corrige o léxico ou a condição e roda de novo.
Cada correção assim torna esses casos incapazes de
testar o sistema, e é para isso que eles existem.
    Holdout    - a metade que se preserva sem olhar. Como o sistema nunca
teve chance de ser moldado por esses casos, é este
número que vale para o relatório. Basta espia-la para
corrigir uma regra e ela vira tune, sem volta.
    Separados - uma caixa para cada metade. A diferença entre elas mostra
o quanto o sistema depende dos casos que já viu.

>>> A divisão é sempre a mesma, em qualquer máquina: cada caso vai para
um lado fixo, e acrescentar casos novos não move ninguém de metade.

>>> Entre metades diferentes, compare recall. Elas têm proporções
diferentes de casos que devem ser bloqueados, e isso desloca a
acurácia e o F1. O recall não se desloca. O relatório imprime um
aviso quando a comparação não é válida.

>>> O conjunto curado não é dividido. Ele foi escrito pelo autor das
próprias regras, e é reportado sempre inteiro e separado, nunca
somado nem promediado com os conjuntos externos.

>>> A avaliação nunca escreve na trilha de auditoria, por desenho.
Rodar centenas de casos encheria a trilha de registros que não são
decisões sobre conteúdo real de ninguém.

>>> Para levar os números para outro lugar, uma planilha ou um script,
use --json na linha de comando. Ele imprime o resultado como dado
estruturado, que um programa lê sem interpretar o texto do
relatório. A tela hoje não exporta: o texto dela serve para copiar,
não para somar.

Exceções ou potenciais problemas:
/////////////////////////////////////////////////////////////////
Se [você pedir uma metade do conjunto curado]
{
    É porque: a tela e a linha de comando recusam, com o mesmo motivo
escrito. Um número rotulado holdout que não é um holdout é
pior que nenhum número.
}

Se [comparar as três demorar]
{
    É porque: roda o mesmo conjunto três vezes. Com "Tune e holdout,
separados" junto, são seis avaliações. Prefira um dataset
menor enquanto estiver explorando.
}

Se [as métricas vierem diferentes das relatadas neste documento]
{
    Então faça: compare o identificador de configuração impresso no
relatório com o registrado aqui.
    É porque: o conjunto, a política ou a ontologia foram alterados. Os
números deste documento valem para a configuração incluída,
e o identificador é o que permite verificar isso em vez de
supor.
}

```

5.7 Tarefa G — Ler a trilha de auditoria

```

Guia de Instruções:
/////////////////////////////////////////////////////////////////
Para [Tarefa G: LER A TRILHA DE AUDITORIA] faça:
Passo 1: Vá à tela Auditoria.
Passo 2: Informe a senha preenchida no assistente de instalação.
Passo 3: Percorra os registros. Cada um traz o conteúdo submetido, o
veredito, as regras e normas disparadas, a origem do registro e

```

```

    o identificador da configuração vigente no momento da decisão.
Passo 4: Para conferir uma decisão antiga, compare o identificador de
    configuração do registro com o da configuração atual.

>>> A origem separa o que veio de uso real do que veio da demonstração.
    Registros marcados como "demo" são do caso roteirizado da Tarefa E,
    não de conteúdo que alguém submeteu.

>>> A senha só separa papéis, ela não é medida de segurança. Quem tem
    acesso à máquina tem acesso ao arquivo da trilha.

>>> O identificador de configuração resume a política, a ontologia
    e as normas que valiam naquele momento. Dois registros com
    identificadores diferentes foram julgados por sistemas diferentes,
    e comparar os vereditos sem notar isso leva a conclusão errada.

>>> A sessão dura 12 horas e não expira por inatividade. Fechar a aba
    não desloga.

Exceções ou potenciais problemas:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Se [a tela Auditoria não existir]
{
    Então faça: leia o aviso que o servidor imprime ao subir. Ele diz
        como habilitá-la.
    É porque: nenhuma senha foi definida na instalação, e sem senha a
        tela não sobe.
}

Se [a trilha estiver vazia]
{
    Então faça: confira de qual pasta o servidor foi subido.
    É porque: ou nenhuma decisão foi registrada ainda, ou o servidor
        está gravando em logs\audit.jsonl de outra pasta. A
        trilha segue o diretório de onde o comando rodou (Tarefa B).
}

Se [você não encontrar registros de uma avaliação que rodou]
{
    É porque: a avaliação não é registrada, por desenho (Tarefa F). A
        trilha guarda decisões sobre conteúdo, não medições de
        desempenho.
}

Se [um registro tiver identificador de configuração diferente dos demais]
{
    É porque: a configuração mudou entre uma decisão e outra.
        É assim que se sabe que dois vereditos não são comparáveis.
}

```

5.8 Tarefa H — Proteger um modelo em código próprio

```

Guia de Instruções:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Para [Tarefa H: PROTEGER UM MODELO EM CÓDIGO PRÓPRIO] faça:
Passo 1: Instale as dependências opcionais.
    $ pip install ollama python-dotenv
Passo 2: Monte a engine híbrida (engine de regras + engine de grafo).
Passo 3: Crie o GuardedAgent com a engine, o cliente do modelo e o logger.
Passo 4: Submeta um conteúdo com process e leia o resultado e a explicação.

from ethical_agent import (
    AuditLogger, CompositeEngine, GuardedAgent, KnowledgeGraphEngine,
    OllamaClient, Policy, RuleBasedEngine,
    default_policy_path, load_default_ontology,
)
engine = CompositeEngine([
    RuleBasedEngine(Policy.from_file(default_policy_path())),
    KnowledgeGraphEngine(load_default_ontology()),
], name="hybrid")
agent = GuardedAgent(engine=engine,
    llm=OllamaClient(model="llama3.2:3b"),
    audit=AuditLogger("logs/audit.jsonl"))
result = agent.process(
    "Collect birthdates of children without asking their parents.")
print(result.status)
print(result.input_verdict.explain())

>>> Para desenvolvimento e testes, troque OllamaClient por MockLLM, que

```

```

dispensa rede.

>>> O caminho passado ao AuditLogger é usado como está. Um caminho
relativo, como no exemplo, resolve contra o diretório de onde o
programa roda.

>>> O GuardedAgent é o orquestrador, não o juiz. Ele encadeia
verificação de entrada, chamada ao modelo e verificação de saída; o
julgamento é todo da engine. Trocar a engine troca o critério sem
tocar no agente.

>>> Passar o AuditLogger é o que faz a decisão ficar registrada. Sem
ele, o agente decide e esquece, e a decisão deixa de ser
auditável, que é justamente o que este sistema se propõe a garantir.

Exceções ou potenciais problemas:
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Se [o serviço do modelo (Ollama) não estiver disponível]
{
  Então faça: use o MockLLM no lugar do OllamaClient.
  É porque: sem o serviço ativo, a geração do modelo falha.
}

Se [você precisar entender por que uma decisão foi tomada]
{
  Então faça: chame verdict.explain(), que devolve as regras e normas
disparadas, os trechos casados, os caminhos de inferência
no grafo e o que foi suprimido por exceção.
}

Se [uma camada da engine falhar]
{
  É porque: a composição é fail-closed. A falha vira DENY, não ALLOW.
  O conteúdo não passa por omissão.
}

```

6 Referências

Ontologia de terceiros. Arora, C.; Sarkar, D. *RelAIEO — Relational AI Ethics Ontology*. Audit4SG. <https://ontology.audit4sg.org/>. Licença GNU GPL v3. Incluída sem alteração em `ontologies/relaieo.ttl`; procedência em `ontologies/PROVENANCE.md`.

Algoritmo da camada de recusa. Chapman, W. W.; Chu, D.; Dowling, J. N. “ConText: An Algorithm for Identifying Contextual Features from Clinical Text”. *BioNLP*, 2007. — Harkema, H.; Dowling, J. N.; Thornblade, T.; Chapman, W. W. “ConText: an algorithm for determining negation, experienter, and temporal status from clinical reports”. *Journal of Biomedical Informatics*, 2009.

Conjuntos de teste. `dataset_beavertails.json` — 220 casos retirados de um conjunto de cerca de 30 mil do benchmark BeaverTails; a seleção não é reproduzível (ver 3.4). `dataset_huggingface_injections.json` — 662 casos, o conjunto inteiro. `dataset.json` — 72 casos escritos pelo próprio autor.

Documentação interna. LICENSE (GNU GPL v3), README.md, AUDIT_GUIDE.pt-BR.md, `ontologies/PROVENANCE.md`, e os arquivos de decisões e de dívida aberta, que guardam as medições que escolheram cada caminho.

7 Apêndice — como reconferir os números deste documento

Todos os comandos rodam a partir da pasta do projeto, na versão 281e7c5.

afirmação	como reconferir
3 regras rígidas e 9 com exceção	<code>abrir policies/core_policy.json</code>
167 conceitos, 18 normas, 54 termos	<code>load_default_ontology()</code> e contar por origem
7 marcas de recusa	<code>abrir frames/refusal_frames.json</code>
os 8 arquivos e o código de configuração	<code>build_configuration(engine)</code> no modo combinado
as medições das Tabelas 1 a 4	<code>ethical-agent --engine {rule,kg,hybrid} eval --dataset ...</code> <code>--half ...</code>
1 306 testes	<code>python -m pytest --collect-only -q</code>
os quatro resultados do Cenário Negativo 1	<code>ethical-agent check --stage output "<texto>"</code>
45 módulos, 9917 linhas, 7,5% de comentário	varredura dos arquivos .py do pacote

Uma ressalva de método: rodar uma verificação grava no registro de auditoria. Para reconferir sem misturar suas provas com o uso real, passe `--audit-log` apontando para um arquivo seu.